

AIN SHAMS UNIVERSITY

Faculty of Engineering

Computer Engineering and Software System

COURSE PROJECT Phase 1

CSE336: Software Design Patterns

Design Report: Class Diagram and Design Patterns

Zakat Misr, Non-Profit Organization Management System

Prepared by:

Youssef Moustafa Elsayed	ID: 22P0047
Nour Eldeen Ahmed Rehab	ID: 22P0040
Mostafa Fouad Ahmed	ID: 22P0077
Ibrahim Shaker	ID: 22P0056
Abdelrahman Mostafa	ID: 22P0053
Mohamed Yasser Khallaf	ID: 22P0245

Presented to Dr. Raghda Essam



Contents

1 Introduction **1**

1.1 Purpose of this report . . . 1
1.2 System in brief . . . 1
1.3 How the design is organised . . . 1
1.4 Notation and code samples . . . 2
1.5 Report outline . . . 2

2 The Class Diagram **3**

2.1 Full diagram . . . 3
2.2 Relationships between classes . . . 3
2.2.1 Inheritance and realization . . . 3
2.2.2 Associations, aggregations and dependencies . . . 3
2.3 Following a donation through the diagram . . . 4

3 Design Patterns **7**

3.1 Singleton . . . 7
3.1.1 Problem . . . 7
3.1.2 Solution . . . 7
3.1.3 Participants . . . 8
3.1.4 Why we chose it . . . 8
3.2 Factory . . . 8
3.2.1 Problem . . . 8
3.2.2 Solution . . . 9
3.2.3 Participants . . . 9
3.2.4 Why we chose it . . . 9
3.3 Decorator . . . 10
3.3.1 Problem . . . 10



3.3.2 Solution . . . 10
3.3.3 Participants . . . 11
3.3.4 Why we chose it . . . 11
3.4 Observer . . . 12
3.4.1 Problem . . . 12
3.4.2 Solution . . . 12
3.4.3 Participants . . . 13
3.4.4 Why we chose it . . . 13
3.5 Strategy . . . 13
3.5.1 Problem . . . 14
3.5.2 Solution . . . 14
3.5.3 Participants . . . 14
3.5.4 Why we chose it . . . 14
3.6 Putting the behavioural pieces together . . . 15
3.7 CRUD Lookup Tables . . . 15
3.7.1 Problem . . . 15
3.7.2 Solution . . . 15
3.7.3 Why we chose it . . . 16
3.8 Model-View-Controller . . . 16
3.9 Summary of pattern choices . . . 16

4 Class Catalogue **17**
4.1 Core package . . . 17
4.2 Donation package . . . 17
4.3 Observer package . . . 18
4.4 Communication package . . . 18
4.5 Lookup package . . . 19
4.6 Users package . . . 19
4.7 Volunteer package . . . 20
4.8 Event package . . . 20



4.9 Beneficiary package 20
4.10 Presentation package 21

5 MVC Architecture 22
5.1 Why MVC 22
5.2 Roles of the classes 22
5.3 Flow of a request 22
5.4 The service layer 22
5.5 One exception worth noting 23

6 CRUD Table 24
6.1 What a CRUD table is 24
6.2 Module chosen 24
6.3 CRUD table for the Volunteer Management module 24
6.4 CRUD table for the Volunteer Management module 24
6.4.1 Reading the table 25
6.4.2 Gaps the table shows 26
6.5 CRUD operations on the lookup tables 26

7 Requirements Coverage 28
7.1 Functional requirements 28
7.2 Non-functional requirements 28

8 Conclusion 30

List of Tables

1.1 Packages of the class diagram 2

2.1 Labelled relationships in the class diagram 3

3.1 Patterns, locations and reasons 16

5.1 MVC roles 22

6.1 CRUD table for the Volunteer Management module 25
6.2 CRUD operations on lookup tables 26

7.1 Functional requirements mapped to classes 28

List of Figures

2.1 Complete class diagram of the Zakat Misr system (Phase 1) 6

3.1 Singleton and Factory region of the class diagram 7
3.2 Donation hierarchy and decorators 10
3.3 Observer region of the class diagram 12
3.4 Strategy region of the class diagram 13
3.5 Lookup table region of the class diagram 15

1. Introduction

1.1 Purpose of this report

The first deliverable of the project was the Software Requirements Specification (SRS) for the Non-Profit Organization Management System. This report is the second part of Phase 1. It takes the requirements written there and turns them into an object-oriented design: a UML class diagram with 46 classes, and the design patterns that give that diagram its structure.

The report has three goals. First, to say what every class in the diagram is and the job it does in the system. Second, to explain each design pattern we used, where it appears in the diagram, and why we picked it over the plainer alternative. Third, to show how the design covers the functional requirements from the SRS, including a CRUD table for one of the modules.

1.2 System in brief

The system is a platform for a charitable organization. We named it Zakat Misr. It handles four areas of work: donations, volunteers, events, and beneficiaries. Four kinds of users interact with it (donors, volunteers, beneficiaries and administrators), and a fifth, non-human actor, the notification service, delivers messages by email, SMS and social media.

1.3 How the design is organised

The 46 classes are grouped into ten packages. Table 1.1 lists them. Most of the packages correspond to one design pattern or one business module, so the package a class lives in usually tells you why it exists.



Table 1.1: Packages of the class diagram

Package	Classes	Content
Core	1	`DatabaseManager` (Singleton)
Donation	8	Factory and Decorator patterns around `Donation`
Observer	4	`Subject`, `Observer`, `DonationService`, `NotificationObserver`
Communication	5	Strategy pattern for Email, SMS and social media
Lookup	4	CRUD-managed lookup tables (`LookupItem` and subclasses)
Users	5	`User` and the four roles
Volunteer	3	Task, assignment and service classes
Event	3	Event, registration and `EventService`
Beneficiary	3	Need, aid distribution and service classes
Presentation	10	MVC views and controllers
Total	**46**	

1.4 Notation and code samples

Names written in `this font` are classes, methods or attributes from the diagram. The short code excerpts in the report are written in Java, because the signatures in the diagram (`List<Observer>`, for example) already follow Java syntax. They only illustrate the idea behind a pattern and are not the final source code.

1.5 Report outline

Chapter 2 presents the full class diagram and the relationships in it. Chapter 3 covers the design patterns one by one. Chapter 4 is the class catalogue, saying what each of the 46 classes is. Chapter 5 explains the MVC structure. Chapter 6 contains the CRUD table. Chapter 7 links the SRS requirements to the classes, and Chapter 8 concludes the report.

2. The Class Diagram

2.1 Full diagram

Figure 2.1, on the last page of this chapter, shows the complete diagram on one page. It is a vector drawing, so it can be zoomed in the PDF without losing detail. The colours group the classes: yellow for the Singleton, green for the donation hierarchy, red for decorators, orange for the Observer pattern, cyan for Strategy, grey for lookup tables, white for users, purple for volunteers, light green for events, and pink for beneficiaries.

2.2 Relationships between classes

The diagram uses five kinds of relationship. Inheritance is drawn with a hollow triangle, interface realization with a dashed line and hollow triangle, aggregation with a hollow diamond, and ordinary associations with a plain line and a label. Dependencies (“uses”, “creates”) are drawn as arrows.

2.2.1 Inheritance and realization

- `ZakatDonation`, `SadaqahDonation`, `InKindDonation` and `DonationDecorator` extend `Donation`.
- `AnonymousDecorator` and `RecurringDecorator` extend `DonationDecorator`.
- `Donor`, `Admin`, `Volunteer` and `Beneficiary` extend `User`.
- `DonationCategory` and `Skill` extend `LookupItem`.
- `DonationService` and `EventService` implement `Subject`.
- `NotificationObserver` implements `Observer`.
- `EmailStrategy`, `SMSStrategy` and `SocialMediaStrategy` implement `CommunicationStrategy`.

2.2.2 Associations, aggregations and dependencies

Table 2.1 lists the labelled relationships. Multiplicities are given where the diagram shows them.

Table 2.1: Labelled relationships in the class diagram

From	Label	To	Kind	Multiplicity
`Donor`	makes	`Donation`	assoc.	1 to 0..*
`DonationDecorator`	wraps	`Donation`	aggreg.	1 wrapped
`DonationService`	uses	`DonationFactory`	dep.	

From	Label	To	Kind	Multiplicity
`DonationFactory`	creates	`Donation`	dep.	
`DonationService`	uses	`DatabaseManager`	dep.	
`DonationService`	notifies	`Observer`	assoc.	1 to 0..*
`NotificationObserver`	triggers	`CommunicationService`	assoc.	
`CommunicationService`	uses	`CommunicationStrategy`	aggreg.	1 to 1
`LookupService`	manages CRUD	`LookupItem`	dep.	
`LookupService`	uses	`DatabaseManager`	dep.	
`Volunteer`	has	`Skill`	assoc.	0..* to 0..*
`VolunteerTask`	requires	`Skill`	assoc.	0..* to 0..*
`Volunteer`	assigned to	`VolunteerAssignment`	assoc.	1 to 0..*
`VolunteerAssignment`	for	`VolunteerTask`	assoc.	0..* to 1
`Beneficiary`	submits	`Need`	assoc.	1 to 0..*
`AidDistribution`	satisfies	`Need`	assoc.	0..* to 1..*
`Donation`	funds	`AidDistribution`	assoc.	0..* to 1
`BeneficiaryService`	creates	`AidDistribution`	dep.	1 to 0..*
`EventService`	manages	`Event`	assoc.	1 to 0..*
`EventService`	creates	`EventRegistration`	dep.	1 to 0..*
`EventRegistration`	for	`Event`	assoc.	0..* to 1
`User`	attends	`Event`	assoc.	0..* to 0..*
Controllers	updates	Views	dep.	
Controllers	calls	Services	dep.	

2.3 Following a donation through the diagram

One scenario touches most of the donation-side classes, so it is a good way to read the diagram.

1. A donor fills in the form. `DonationController.submitDonation` receives the type, the donor and the amount.
2. The controller calls `DonationService.processDonation`.
3. The service asks `DonationFactory.createDonation` for the right `Donation` subclass.
4. If the donor asked to stay anonymous or to give monthly, the donation is wrapped in an `AnonymousDecorator` or a `RecurringDecorator`.
5. The service gets the single `DatabaseManager` through `getInstance()` and saves the donation with `executeQuery`.
6. The service calls `notifyObservers`. Every attached observer, here `NotificationObserver`, gets `update(d)`.
7. `NotificationObserver` reads `preferredChannel` from the donor, picks Email, SMS or Social Media, sets it on `CommunicationService`, and calls `notifyUser`.
8. The chosen strategy sends the acknowledgement.
9. Back in the controller, `DonationView.displayReceipt` shows the receipt to the donor.



Steps 3, 4, 5, 6 and 7 are where the Factory, Decorator, Singleton, Observer and Strategy patterns do their work. The next chapter takes them in turn.

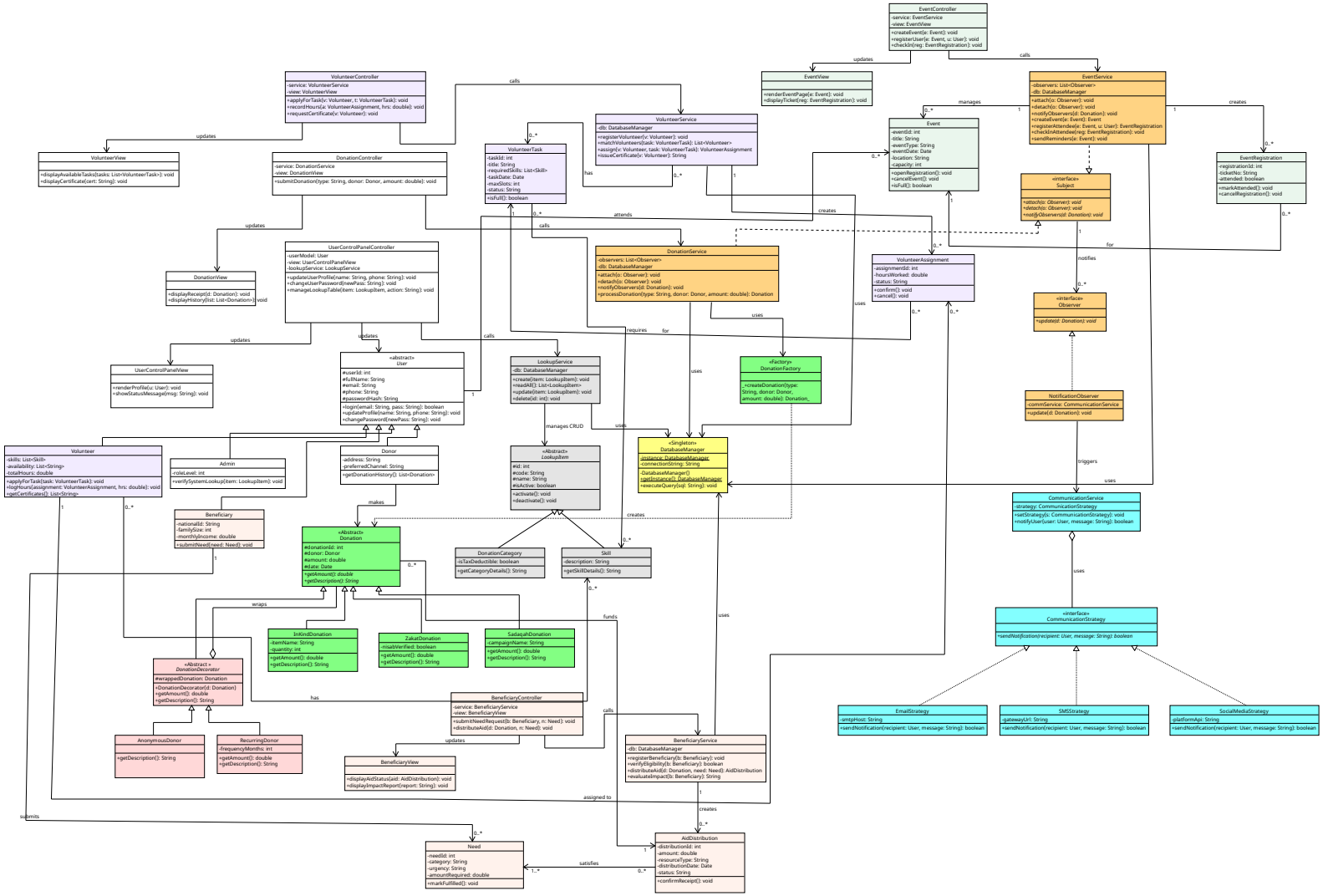


Figure 2.1: Complete class diagram of the Zakat Misr system (Phase 1)

3. Design Patterns

We used five GoF patterns (Singleton, Factory, Decorator, Observer, Strategy), one architectural pattern (MVC), and a generic CRUD scheme for lookup tables. For each one this chapter states the problem in our system, the classes involved, and the reason we chose it. We also say what the pattern costs us, because every pattern adds some classes and some indirection.

3.1 Singleton

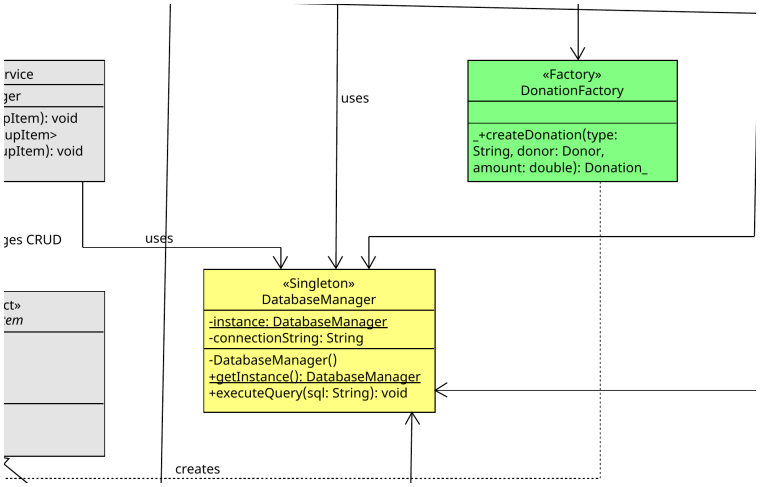


Figure 3.1: Singleton and Factory region of the class diagram

3.1.1 Problem

Five different services (`DonationService`, `LookupService`, `VolunteerService`, `EventService` and `BeneficiaryService`) all need to talk to the database. If each one opened its own connection, the system would hold five or more connections for no reason, and a change to the connection string would have to be repeated in several places. The SRS also asks for a response time under 200 ms and for 500 concurrent users (requirements 2.1 and 2.3), so wasting connections is not something we can afford.

3.1.2 Solution

`DatabaseManager` is a Singleton. Its constructor is private, it keeps the only instance in the static field `instance`, and it hands that instance out through `getInstance()`. The connection string is stored once in the class. Services call `DatabaseManager.getInstance().executeQuery(sql)` and never create the object themselves.



```
public class DatabaseManager {
    private static DatabaseManager instance;
    private String connectionString;

    private DatabaseManager() { /* read config, open connection */ }

    public static synchronized DatabaseManager getInstance() {
        if (instance == null) {
            instance = new DatabaseManager();
        }
        return instance;
    }

    public void executeQuery(String sql) { /* run the statement */ }
}
```

3.1.3 Participants

Role	Class	Notes
Singleton	`DatabaseManager`	Private constructor, static `instance`, public `getInstance()`
Clients	The five service classes	Each one holds a `db` attribute that points to the shared instance

3.1.4 Why we chose it

A database connection is the textbook case for this pattern: there should be exactly one, and everything needs access to it. A global variable would also give shared access, but it would not stop another class from creating a second manager. Making the constructor private removes that possibility at compile time. The method is marked `synchronized` in the sample because the application is a web system and two requests can arrive at the same moment.

The cost is that Singletons are hard to replace in unit tests, since the services reach for a global instance. We accept this for Phase 1. If testing becomes painful, the services can receive the manager through their constructors while the Singleton still creates it.

3.2 Factory

3.2.1 Problem

The organization accepts three kinds of donation. Zakat has to record whether the nisab (the minimum wealth threshold) was verified. Sadaqah belongs to a named campaign. An in-kind donation is an item with a quantity rather than money. Without a factory, the code that handles a form submission would need an `if` chain that knows every constructor. Every new type, such as Waqf, would mean editing that code in several places.

3.2.2 Solution

`DonationFactory` exposes one static method, `createDonation(type, donor, amount)`. It looks at the `type` string and returns a `ZakatDonation`, `SadaqahDonation` or `InKindDonation`. The caller only ever sees the abstract type `Donation`.

```
public class DonationFactory {
    public static Donation createDonation(String type, Donor donor, double amount) {
        switch (type.toLowerCase()) {
            case "zakat":   return new ZakatDonation(donor, amount);
            case "sadaqah": return new SadaqahDonation(donor, amount);
            case "inkind":  return new InKindDonation(donor, amount);
            default: throw new IllegalArgumentException("Unknown type: " + type);
        }
    }
}
```

3.2.3 Participants

Role	Class	Notes
Factory	`DonationFactory`	Static creation method
Abstract product	`Donation`	Declares `getAmount()` and `getDescription()`
Concrete products	`ZakatDonation`, `SadaqahDonation`, `InKindDonation`	Each adds its own fields
Client	`DonationService`	Calls the factory inside `processDonation`

3.2.4 Why we chose it

The choice of subclass depends on data that arrives at run time (the donor picks a type on the form), so the decision has to be made in code, and it should be made in only one place. The factory is that place. `DonationService` stays short and does not change when a new donation type is added; only the factory and the new subclass do.

To be accurate about the pattern name: because the method is static and sits in its own class, this is a simple (static) factory, a close relative of the GoF Factory Method. We kept it static because the factory carries no state and nobody needs to subclass it. The price is that adding a type still means editing the `switch`. That is acceptable for three types.

3.3 Decorator

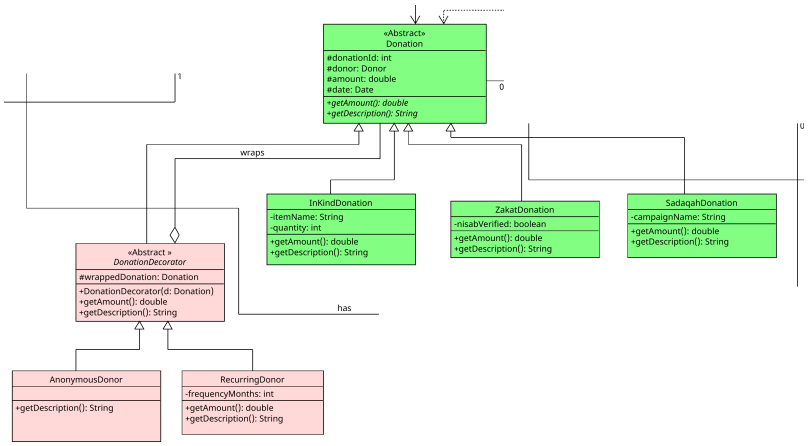


Figure 3.2: Donation hierarchy and decorators

3.3.1 Problem

Donations can carry extra behaviour that has nothing to do with their type. A donor may want to give anonymously. A donor may want the same amount to repeat every few months. Both options can apply to a Zakat, a Sadaqah or an in-kind gift, and they can apply together. Handling this with subclasses would give classes like `AnonymousZakatDonation`, `RecurringSadaqahDonation` and `AnonymousRecurringInKindDonation`. With three types and two options that is already nine classes, and it doubles each time we add an option.

3.3.2 Solution

`DonationDecorator` extends `Donation` and also holds a `Donation` (`wrappedDonation`). By default it forwards `getAmount()` and `getDescription()` to the wrapped object. The two concrete decorators change only what they need to:

- `AnonymousDecorator` overrides `getDescription()` so the donor is shown as "Anonymous".
- `RecurringDecorator` stores `frequencyMonths` and adds the recurring schedule to the description. It also overrides `getAmount()` so the recurring figure can be reported on its own.

Because a decorator is itself a `Donation`, decorators can be stacked:

```
Donation d = DonationFactory.createDonation("zakat", donor, 500.0);
d = new RecurringDecorator(d, 3);   // every 3 months
d = new AnonymousDecorator(d);      // hide the donor name
System.out.println(d.getDescription());
```

3.3.3 Participants

Role	Class	Notes
Component	`Donation`	Abstract class shared by products and decorators
Concrete components	`ZakatDonation`, `SadaqahDonation`, `InKindDonation`	The objects being wrapped
Decorator	`DonationDecorator`	Holds `wrappedDonation`, delegates by default
Concrete decorators	`AnonymousDecorator`, `RecurringDecorator`	Add one behaviour each

3.3.4 Why we chose it

Inheritance fixes behaviour at compile time, while a donor's choices are made at run time and can be combined in any order. Decorator lets us build the combination when the donation is created, and it keeps one class per option. Adding a third option later, for example a "gift aid" marker, means writing one new class and touching nothing else.

Two things to watch. A long chain of decorators is harder to debug. And since `Donation` is an abstract class and not an interface, every decorator inherits the fields `donationId`, `donor`, `amount` and `date` even though it forwards everything to the wrapped object. We live with that because it lets the rest of the system treat a decorated donation exactly like a plain one.

3.4 Observer

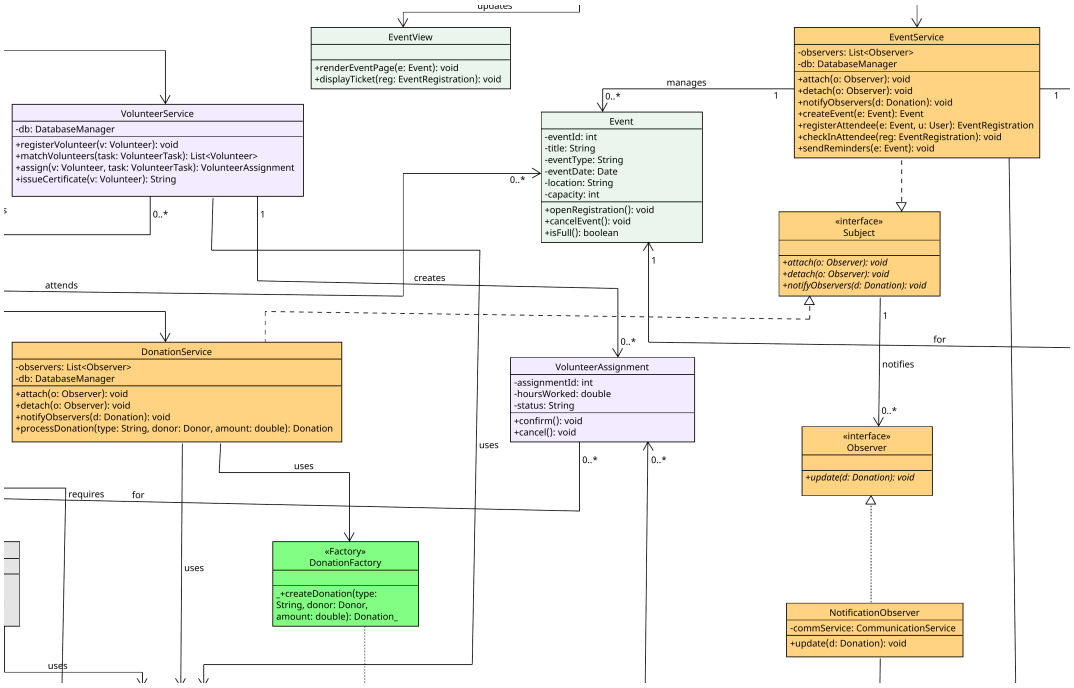


Figure 3.3: Observer region of the class diagram

3.4.1 Problem

When a donation is saved, several things should happen: the donor gets an acknowledgement (requirement 1.7), and later we may want a statistics update, an audit entry or a message to the finance team. If `DonationService.processDonation` called each of these directly, it would depend on every one of them, and adding another would mean editing the service.

3.4.2 Solution

`Subject` declares `attach`, `detach` and `notifyObservers`. `DonationService` implements it and keeps a `List<Observer>`. After it saves a donation, it loops over the list and calls `update(d)` on each observer. `NotificationObserver` is the one observer we have in Phase 1. It reads the donor's preferred channel and starts the notification (see the Strategy section).

```
public Donation processDonation(String type, Donor donor, double amount) {
    Donation d = DonationFactory.createDonation(type, donor, amount);
    DatabaseManager.getInstance().executeQuery(/* INSERT donation */);
    notifyObservers(d);
    return d;
}

public void notifyObservers(Donation d) {
    for (Observer o : observers) {
        o.update(d);
```

```
    }
}
```

3.4.3 Participants

Role	Class	Notes
Subject	`Subject` (interface)	`attach`, `detach`, `notifyObservers`
Concrete subjects	`DonationService`, `EventService`	Both keep an `observers` list
Observer	`Observer` (interface)	Single method `update(d: Donation)`
Concrete observer	`NotificationObserver`	Sends the donor a message

3.4.4 Why we chose it

The service that saves a donation should not know who cares about it. With Observer, the service only knows the `Observer` interface. New reactions are added with `attach()` and removed with `detach()`, without changing `processDonation`. This also fits the SRS well: requirement 1.7 asks for automatic acknowledgements, and that is exactly an event followed by a reaction.

The main risk is that observers run in the same thread by default, so a slow SMS gateway could delay the donor's receipt page. If that shows up when we test against the 200 ms target, the notification step can be moved to a background task without changing the interfaces.

3.5 Strategy

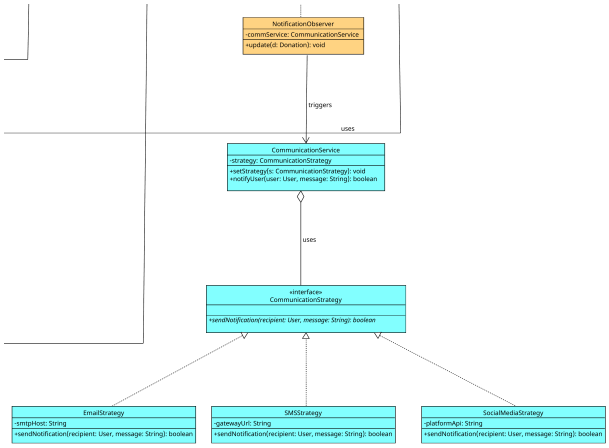


Figure 3.4: Strategy region of the class diagram

3.5.1 Problem

The SRS says communication must work through Email, SMS and Social Media (sections 1.2 and 1.7). The three channels need different technical details: an SMTP host, an SMS gateway URL, a social platform API. Putting all of that in one class behind a chain of `if` statements would make the class large and hard to test, and a fourth channel such as WhatsApp would mean reopening it.

3.5.2 Solution

`CommunicationStrategy` is an interface with one method, `sendNotification(recipient, message)`. `EmailStrategy`, `SMSStrategy` and `SocialMediaStrategy` each implement it using their own settings. `CommunicationService` is the context. It holds the current strategy, lets callers change it with `setStrategy`, and delegates in `notifyUser`.

```
public void update(Donation d) {
    Donor donor = d.getDonor();
    switch (donor.getPreferredChannel()) {
        case "sms":    commService.setStrategy(new SMSStrategy());         break;
        case "social": commService.setStrategy(new SocialMediaStrategy()); break;
        default:       commService.setStrategy(new EmailStrategy());
    }
    commService.notifyUser(donor, "Thank you for your donation: " + d.getDescription());
}
```

3.5.3 Participants

Role	Class	Notes
Context	`CommunicationService`	Holds `strategy`, exposes `notifyUser`
Strategy	`CommunicationStrategy` (interface)	`sendNotification`
Concrete strategies	`EmailStrategy`, `SMSStrategy`, `SocialMediaStrategy`	Own `smtpHost`, `gatewayUrl`, `platformApi`
Client	`NotificationObserver`	Chooses the strategy from `preferredChannel`

3.5.4 Why we chose it

The channel is a choice the donor makes, and it can differ for every user, so the algorithm has to be swappable at run time. Strategy does this with no conditionals inside the sending code. Each channel can be tested alone, and adding a channel means one new class that implements the interface. Strategy and Observer also work well together here: Observer decides *when* to notify, Strategy decides *how*.

3.6 Putting the behavioural pieces together

Observer, Strategy and Factory are connected through `DonationService`. The service creates the object (Factory), saves it through the shared manager (Singleton), announces it (Observer), and the observer picks a channel (Strategy). No class in this chain depends on a concrete class further down; they depend on `Donation`, `Observer` and `CommunicationStrategy`. That is why each piece can be replaced independently.

3.7 CRUD Lookup Tables

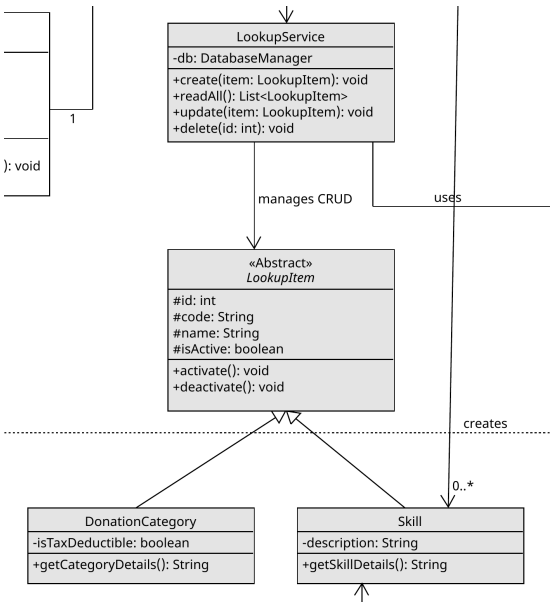


Figure 3.5: Lookup table region of the class diagram

3.7.1 Problem

Some data is a short list of values that administrators need to change without a code release: donation categories (with a tax-deductible flag) and volunteer skills. If these were hard-coded as enums, a new skill would need a developer.

3.7.2 Solution

`LookupItem` is an abstract class with the fields every lookup row shares: `id`, `code`, `name` and `isActive`, plus `activate()` and `deactivate()`. `DonationCategory` and `Skill` extend it with their own fields. One class, `LookupService`, offers `create`, `readAll`, `update` and `delete` for any `LookupItem`, using the shared `DatabaseManager`. The administrator reaches it through `UserControlPanelController.manageLookupTable`.

3.7.3 Why we chose it

Putting the common fields in one parent and the four operations in one service means a new lookup table (say, event types) needs only a small subclass. We did not write a separate maintenance screen and service for each list. The `isActive` flag lets an administrator retire a value without deleting rows that old records still point to. Chapter 6 shows which operations each user task performs on these tables.

This part is not a GoF pattern. It is a standard way of structuring maintenance data, and we list it here because the project description asks for it.

3.8 Model-View-Controller

MVC is explained in Chapter 5, since it touches the whole system and not one group of classes. In short: the domain classes (`User` and its subclasses, `Donation`, `Event`, `VolunteerTask` and so on) are the models, five `...View` classes show data, and five `...Controller` classes receive user actions and call the services.

3.9 Summary of pattern choices

Table 3.1: Patterns, locations and reasons

Pattern	Main classes	Reason for choosing it
Singleton	`DatabaseManager`	One shared connection and one place for the connection string
Factory	`DonationFactory`	Creation of three donation types in one place
Decorator	`DonationDecorator` and two subclasses	Combine anonymous and recurring options without a subclass for each combination
Observer	`Subject`, `Observer`, `DonationService`, `NotificationObserver`	Donation service does not need to know who reacts to a new donation
Strategy	`CommunicationService` and three strategies	Email, SMS and social media chosen per donor at run time
MVC	Models, five views, five controllers	Separate screens, input handling and business data
CRUD lookup	`LookupItem`, `LookupService`	Administrators maintain categories and skills without code changes

4. Class Catalogue

This chapter lists all 46 classes, grouped by package and numbered as in the specification, and says in a sentence or two what each one is. Where the diagram labels the two decorators `AnonymousDonor` and `RecurringDonor`, we use `AnonymousDecorator` and `RecurringDecorator`, the names in the written specification, because the classes wrap a donation and are not donors.

4.1 Core package

#	Class	Kind	What it is
1	`DatabaseManager`	*Singleton*	The only class that talks to the database. Other classes request the shared instance and run SQL through it.

4.2 Donation package

#	Class	Kind	What it is
2	`DonationFactory`	*Factory*	Builds the correct kind of donation from a type name, so callers never use the concrete constructors.
3	`Donation`	*abstract class*	The common parent of every donation, and the component that decorators wrap. It stands for who gave, how much, and when.
4	`ZakatDonation`	*extends Donation*	A Zakat payment, which records whether the nisab threshold was checked before it was accepted.
5	`SadaqahDonation`	*extends Donation*	A voluntary donation, linked to a named campaign.
6	`InKindDonation`	*extends Donation*	A donation of goods, such as food or clothes, given as an item and a quantity. Its amount is an estimated value.
7	`DonationDecorator`	*abstract class, extends Donation*	The base class for decorators. It wraps another donation and passes everything through to it.
8	`AnonymousDecorator`	*extends DonationDecorator*	Hides the donor's identity in the description, for donors who asked to give anonymously.



#	Class	Kind	What it is
9	`RecurringDecorator`	*extends DonationDecorator*	Marks a donation as repeating every few months and adds that schedule to the description.

4.3 Observer package

#	Class	Kind	What it is
10	`Subject`	*interface*	The role of something that can be watched. Observers register with it and are told when something happens.
11	`Observer`	*interface*	The role of something that wants to be told about a new donation.
12	`DonationService`	*implements Subject*	The main business class for donations. It creates the donation through the factory, saves it through the database manager, and then notifies its observers.
13	`NotificationObserver`	*implements Observer*	Reacts to a new donation by contacting the donor through the channel the donor prefers.

4.4 Communication package

#	Class	Kind	What it is
14	`CommunicationService`	*Strategy context*	Sends a message to a user with whichever strategy is currently set, without knowing which channel it is.
15	`CommunicationStrategy`	*interface*	The contract every channel follows: send a message to a recipient and report whether it went out.
16	`EmailStrategy`	*implements CommunicationStrategy*	Sends messages by email through an SMTP server.
17	`SMSStrategy`	*implements CommunicationStrategy*	Sends text messages through an SMS gateway.
18	`SocialMediaStrategy`	*implements CommunicationStrategy*	Sends messages through a social platform's API.

4.5 Lookup package

#	Class	Kind	What it is
19	`LookupItem`	*abstract class*	The common parent for rows of lookup tables. Every row has an id, a short code, a display name and an active flag.
20	`DonationCategory`	*extends LookupItem*	A category a donation can belong to, such as Zakat al-Mal or Sadaqah Jariyah, including whether it is tax deductible.
21	`Skill`	*extends LookupItem*	A skill a volunteer can have and a task can require, such as first aid or teaching.
22	`LookupService`	*service*	Creates, reads, updates and deletes any lookup item using the shared database manager.

4.6 Users package

#	Class	Kind	What it is
23	`User`	*abstract class, MVC model*	The shared data and behaviour of everyone who logs in: account details, login and password handling. The roles are its subclasses.
24	`Donor`	*extends User*	Someone who gives. Their preferred channel decides how they are notified.
25	`Admin`	*extends User*	A staff member who manages the system and its lookup data. Its role level supports role-based access control.
26	`Volunteer`	*extends User*	A person who offers time. Holds skills, availability and the running total of service hours.
27	`Beneficiary`	*extends User*	A person or family receiving help, with the details needed for the eligibility check.

4.7 Volunteer package

#	Class	Kind	What it is
28	`VolunteerTask`	*MVC model*	A piece of work that needs volunteers, with a date, a limit on slots and the skills it requires.
29	`VolunteerAssignment`	*MVC model*	Links one volunteer to one task and records the hours worked and the state of the assignment.
30	`VolunteerService`	*service*	The business logic of the volunteer module: registration, finding suitable volunteers, assigning them, and issuing certificates.

4.8 Event package

#	Class	Kind	What it is
31	`Event`	*MVC model*	A fundraiser or community event with a date, a place and a capacity.
32	`EventRegistration`	*MVC model*	One person's place at one event, with the ticket number used at check-in.
33	`EventService`	*implements Subject*	The business logic for events: creating them, registering attendees, checking people in, and sending reminders.

4.9 Beneficiary package

#	Class	Kind	What it is
34	`Need`	*MVC model*	A request for help from a beneficiary, with a category, an urgency and the amount required.
35	`AidDistribution`	*MVC model*	A record of aid given out. It links a donation to a need, and the beneficiary confirms receipt.
36	`BeneficiaryService`	*service*	The business logic of the beneficiary module: registering people, checking eligibility, distributing aid, and judging its impact.

4.10 Presentation package

#	Class	Kind	What it is
37	`DonationView`	*MVC view*	Shows a receipt for one donation and the donation history of a donor.
38	`DonationController`	*MVC controller*	Receives a donation request from the screen and passes it to the donation service.
39	`UserControlPanelView`	*MVC view*	Draws the profile page and shows status messages such as “password changed”.
40	`UserControlPanelController`	*MVC controller*	Handles profile and password changes, and lets administrators manage lookup tables from the same panel.
41	`VolunteerView`	*MVC view*	Lists the tasks a volunteer can apply for and shows a generated certificate.
42	`VolunteerController`	*MVC controller*	Handles task applications, hour logging and certificate requests.
43	`EventView`	*MVC view*	Renders the event page and the ticket a registered attendee receives.
44	`EventController`	*MVC controller*	Handles event creation, user registration and check-in at the door.
45	`BeneficiaryView`	*MVC view*	Shows the status of aid a beneficiary is due to receive and the impact report.
46	`BeneficiaryController`	*MVC controller*	Handles need requests and the distribution of aid.

5. MVC Architecture

5.1 Why MVC

The SRS asks for a web application used by four groups of people through modern browsers (requirement 2.4) and built with standard object-oriented frameworks (requirement 2.8). Each group sees a different set of screens, but the rules behind them are shared. MVC keeps the two apart: a screen can change without touching the business rules, and the rules can change without rewriting the screen.

5.2 Roles of the classes

Table 5.1: MVC roles

Layer	Classes
Model	`User`, `Donor`, `Admin`, `Volunteer`, `Beneficiary`, `Donation` and subclasses, `VolunteerTask`, `VolunteerAssignment`, `Event`, `EventRegistration`, `Need`, `AidDistribution`
View	`DonationView`, `UserControlPanelView`, `VolunteerView`, `EventView`, `BeneficiaryView`
Controller	`DonationController`, `UserControlPanelController`, `VolunteerController`, `EventController`, `BeneficiaryController`
Service (business logic)	`DonationService`, `VolunteerService`, `EventService`, `BeneficiaryService`, `LookupService`, `CommunicationService`
Infrastructure	`DatabaseManager`

5.3 Flow of a request

A request moves in one direction through the layers. The user acts on a screen, the controller receives the action, the controller calls a service, the service works with models and the database, and the controller then asks the view to display the result. Views do not call services, and models do not know about views. The controller arrows in the diagram are labelled *updates* (to the view) and *calls* (to the service) for this reason.

5.4 The service layer

The specification adds a service class between controllers and models in every module. We chose this so that controllers stay thin. For example, `VolunteerController.applyForTask`



does not check whether the task is full or whether the volunteer has the right skills. That logic belongs to `VolunteerService` and `VolunteerTask.isFull()`, where it can be reused by the administrator's automatic matching without going through a screen.

5.5 One exception worth noting

`UserControlPanelController` holds a `User` model directly and also a `LookupService`. It serves two jobs: a normal user editing a profile, and an administrator editing lookup data. We kept them together because both belong to the user's control panel in the SRS, but if the panel grows it should be split.

6. CRUD Table

6.1 What a CRUD table is

A CRUD table (also called a CRUD matrix) links the functions of an application to the data it stores. The rows are functions or user tasks, the columns are entities, and each cell contains the letters of the operations that function performs on that entity: **C**reate, **R**ead, **U**pdate, **D**elete. An empty cell means the function does not touch that entity. When the matrix is used for permissions, the columns are roles instead of entities.

The table is useful for two reasons. It shows what each function really needs from the data, which exposes missing functions: an entity with a C but no D means nothing can ever be removed, and an entity with no R is never shown to anyone. It also helps divide the work, because every column maps to a model class and every row to a controller method.

6.2 Module chosen

We chose the **Volunteer Management module**. It has the most interaction between entities (volunteers, skills, tasks and assignments), it involves two actors with different rights (volunteer and administrator), and it also uses the lookup table for skills, so one table can show both the normal flow and the lookup maintenance.

6.3 CRUD table for the Volunteer Management module

The rows come from requirements 1.3 and 1.6 of the SRS and from the operations of `VolunteerService`, `VolunteerController` and `Volunteer`. The columns are the four entities of the module (Task stands for `VolunteerTask` and Assignment for `VolunteerAssignment`). When a function reads an entity only to check something, we still mark it with R, because the read is a real access.

6.4 CRUD table for the Volunteer Management module

The rows come from requirements 1.3 and 1.6 of the SRS and from the operations of `VolunteerService`, `VolunteerController`, `Volunteer`, `VolunteerAssignment` and `LookupService`. The columns are the four entities of the module (Task stands for `VolunteerTask` and Assignment for `VolunteerAssignment`). A letter is written only when the function reads or changes that entity's data. A check done before an action (for example, testing whether a skill is still in use) counts as a read, and a side effect (for example, reserving a slot on a task) counts as an update.



Table 6.1: CRUD table for the Volunteer Management module

#	Function	Actor	Volunteer	Skill	Task	Assignment
F1	Register as a volunteer	Volunteer	C			
F2	Update profile and password	Volunteer	U			
F3	Add availability	Volunteer	U			
F4	Add a skill to my profile	Volunteer	U	R		
F5	Create a skill (lookup list)	Admin		C		
F6	Edit or deactivate a skill	Admin		U		
F7	Delete a skill	Admin	R	D	R	
F8	Create a volunteer task	Admin		R	C	
F9	Browse available tasks	Volunteer		R	R	
F10	Apply for a task / assign a volunteer	Volunteer / Admin	R		R, U	C
F11	Match volunteers to a task	Admin	R		R	
F12	Confirm or cancel an assignment	Admin / Volunteer			U	U
F13	Log service hours	Volunteer	U			R, U
F14	View own volunteering history	Volunteer	R		R	R
F15	Issue a volunteer certificate	Admin	R			
F16	Remove a volunteer account	Admin	D		U	U, D

6.4.1 Reading the table

- **F1 to F3.** Registration only creates the `Volunteer` record (name, e-mail, phone, password). No skills, tasks or assignments exist yet, so those columns stay empty. Profile and availability changes update the same record.
- **F4.** Skills are added after registration. The volunteer chooses from the active skills in the lookup table, so Skill is read, and the chosen skill is stored on the volunteer, so Volunteer is updated.
- **F5 to F7.** These are the lookup operations. They map onto `LookupService.create`, `update` and `delete` (Table 6.2). A skill is referenced by volunteers (their skills) and by tasks (their required skills), so deleting one means checking both; this is why F7 reads Volunteer and Task. Deactivating a skill (F6) is the safer way to retire it.
- **F8 and F9.** Creating a task reads the skill list, because the administrator picks the required skills. Browsing tasks reads the tasks together with their required skills.
- **F10.** Applying for a task and being assigned to one use the same method, `VolunteerService.assign`, so they have the same effect on the data. The method reads the volunteer to check the skills, creates the `VolunteerAssignment`, and updates the task because it reserves a slot (`reserveSlot`), which `isFull()` depends on. The difference between the two is only the actor.
- **F11.** Matching compares each volunteer's skills with the task's required skills. It reads both entities and changes nothing.
- **F12.** Confirming changes the status of the assignment. Cancelling also releases the slot, which updates the task.



- **F13.** Logging hours updates two entities: the hours worked on the assignment (which is read first, since new hours are added to the old ones) and `totalHours` on the volunteer.
- **F15.** A certificate is generated from the volunteer's `totalHours`. It is returned as a `String` and not stored, so no create appears and no other entity is touched.
- **F16.** Removing a volunteer cancels all of their assignments (an update on the assignments and on the tasks, since slots are released) and then deletes them, together with the volunteer.

6.4.2 Gaps the table shows

Building the table showed a few things about the current design.

- **No task maintenance.** A task can be created (F8) but there is no function to edit, close or delete it. If the project needs these, they must be added to `VolunteerService` and then to the table (Task U and Task D, with a read of Assignment to avoid deleting a task that has volunteers). Otherwise they should be listed as out of scope.
- **No in-use check when deleting a skill.** F7 shows that Volunteer and Task should be read first, but `LookupService.delete` does not do this check at the moment. Until it does, administrators should deactivate skills and not delete them.
- **Assignments are cancelled, not deleted, except when a volunteer is removed.** Cancelling (F12) keeps the record. Removing a volunteer (F16) deletes their assignments, so the hours they logged disappear from the record. If hours must be kept for reports, the assignments should be kept and marked cancelled instead.

6.5 CRUD operations on the lookup tables

Table 6.2 shows how the generic `LookupService` supports the CRUD operations for the two lookup tables, and which role performs them.

Table 6.2: CRUD operations on lookup tables

Operation	Method	Effect	Entities	Who
Create	`create(item)`	Inserts a new row with `isActive` true	Skill, DonationCategory	Admin
Read	`readAll()`	Returns all rows, shown in drop-down lists and the admin table	Skill, DonationCategory	Everyone
Update	`update(item)`	Changes name, code or type-specific fields; `activate()` and `deactivate()` change the flag	Skill, DonationCategory	Admin
Delete	`delete(id)`	Removes a row that has no references	Skill, DonationCategory	Admin

All four operations reach the screen through the `manageLookupTable(item, action)` method of `UserControlPanelController`, where `action` is one of the four operation names. The controller



forwards the call to `LookupService`, which runs the SQL through the `DatabaseManager`. Only users with an `Admin` role may call it.

7. Requirements Coverage

7.1 Functional requirements

Table 7.1 connects each functional requirement from the SRS to the classes that carry it out and the patterns involved.

Table 7.1: Functional requirements mapped to classes

Req.	Requirement (short)	Classes and patterns
1.1	Create accounts, log in securely	`User` (`login`, `changePassword`) and its four subclasses; `UserControlPanelController` and view
1.2	Online and offline contributions, receipts	`DonationController`, `DonationService`, `DonationFactory`, three donation types (`InKindDonation` covers goods handed over in person), `DonationView.displayReceipt`; Decorator for anonymous and recurring gifts
1.3	Skills, availability, service hours	`Volunteer`, `Skill`, `VolunteerAssignment`, `VolunteerController.recordHours`
1.4	Create, schedule and publish events	`Event`, `EventService.createEvent`, `EventController`, `EventView`
1.5	Register needs, track aid	`Beneficiary.submitNeed`, `Need`, `AidDistribution`, `BeneficiaryService`, `BeneficiaryView`
1.6	Assign volunteers by automatic matching	`VolunteerService.matchVolunteers` and `assign`; `VolunteerTask.requiredSkills`
1.7	Automatic reminders and acknowledgements	Observer (`DonationService`, `NotificationObserver`), Strategy (`CommunicationService` and three strategies), `EventService.sendReminders`
1.8	Impact reports and certificates	`BeneficiaryService.evaluateImpact`, `BeneficiaryView.displayImpactReport`, `VolunteerService.issueCertificate`, `VolunteerView.displayCertificate`

7.2 Non-functional requirements

Class diagrams cannot show most non-functional requirements, so only some of them can be tied to the design.

- **2.1, 2.3, 2.5 (speed, concurrent users, growth).** The Singleton avoids repeated connection set-up. Services are separate from controllers, so they can be scaled or cached later. Slow notifications may need a background worker, as noted in the Observer section.
- **2.6 (secure authentication, role-based access).** `User.passwordHash` avoids storing raw passwords, and `Admin.roleLevel` gives a field to test permissions against.



- **2.4 and 2.8 (browsers, object-oriented frameworks).** The MVC split in Chapter 5 fits common web frameworks.
- **2.2, 2.7, 2.9, 2.10.** These concern deployment, schedule and law. They are handled in implementation and project planning, not in the class diagram.

8. Conclusion

The design in this report turns the eight functional requirements of the SRS into 46 classes in ten packages. Five GoF patterns carry the main behaviour: the Singleton gives one database connection, the Factory creates donation types, the Decorator adds anonymous and recurring options, the Observer reacts to new donations, and the Strategy chooses the notification channel. MVC separates screens, controllers and business logic, and the lookup tables let administrators change categories and skills without a developer. The CRUD table for the Volunteer Management module shows how each user function touches the data.

With this class diagram as a base, Phase 2 will add further patterns and implement the core modules, as shown in the time plan of the SRS.